

Record ed enum

Record ed enum

Come rappresentare dati compatti e insiemi di valori fissi.

Due strumenti per dati semplici

Java offre strumenti che rendono più comodo rappresentare alcuni dati.

In questa pagina vediamo:

- `enum` , per valori fissi
- `record` , per oggetti semplici e immutabili

Sono concetti più moderni rispetto alle basi, ma molto utili quando il modello dei dati è chiaro.

enum

Un `enum` rappresenta un insieme chiuso di valori possibili.

Esempio:

```
public enum Giorno {  
    LUNEDI,  
    MARTEDI,  
    MERCOLEDI,  
    GIOVEDI,  
    VENERDI,  
    SABATO,  
    DOMENICA  
}
```

Una variabile di tipo `Giorno` può contenere solo uno di questi valori.

```
Giorno oggi = Giorno.LUNEDI;
```

Perche usare enum

Senza enum potresti usare stringhe:

```
String stato = "pagato";
```

Ma potresti anche sbagliare:

```
String stato = "pagatto";
```

Con un enum, Java controlla i valori validi.

```
public enum StatoOrdine {  
    NUOVO,  
    PAGATO,  
    SPEDITO,  
    CONSEGNATO  
}
```

Uso:

```
StatoOrdine stato = StatoOrdine.PAGATO;
```

enum con switch

```
StatoOrdine stato = StatoOrdine.SPEDITO;

switch (stato) {
    case NUOVO:
        System.out.println("Ordine creato");
        break;
    case PAGATO:
        System.out.println("Pagamento ricevuto");
        break;
    case SPEDITO:
        System.out.println("Ordine in viaggio");
        break;
    case CONSEGNATO:
        System.out.println("Ordine consegnato");
        break;
}
```

Gli enum sono ottimi quando hai un elenco limitato e conosciuto di scelte.

record

Un `record` serve a rappresentare dati semplici.

```
public record Prodotto(String nome, double prezzo) {
}
```

Questa riga crea automaticamente:

- campi privati e finali
- costruttore
- metodi per leggere i valori
- `toString`
- `equals`
- `hashCode`

Usare un record

```
public class EsempioRecord {  
    public static void main(String[] args) {  
        Prodotto prodotto = new Prodotto("Quaderno", 2.50);  
  
        System.out.println(prodotto.nome());  
        System.out.println(prodotto.prezzo());  
        System.out.println(prodotto);  
    }  
}
```

Output:

```
Quaderno  
2.5  
Prodotto[nome=Quaderno, prezzo=2.5]
```

I metodi di lettura hanno lo stesso nome dei campi: `nome()` e `prezzo()`.

Record immutabili

Un record è pensato per dati immutabili: dopo la creazione, non cambi i campi.

Non fai:

```
prodotto.prezzo = 3.00; // non valido
```

Se ti serve un prodotto con prezzo diverso, crei un nuovo record.

Quando usarli

Usa `enum` quando un valore può essere scelto da un insieme fisso:

- giorni della settimana

- stati di un ordine
- livelli di priorita

Usa `record` quando vuoi trasportare dati semplici senza scrivere una classe lunga:

- prodotto con nome e prezzo
- punto con x e y
- persona con nome ed eta

Se l'oggetto deve cambiare spesso o avere molta logica interna, una classe normale puo essere piu adatta.